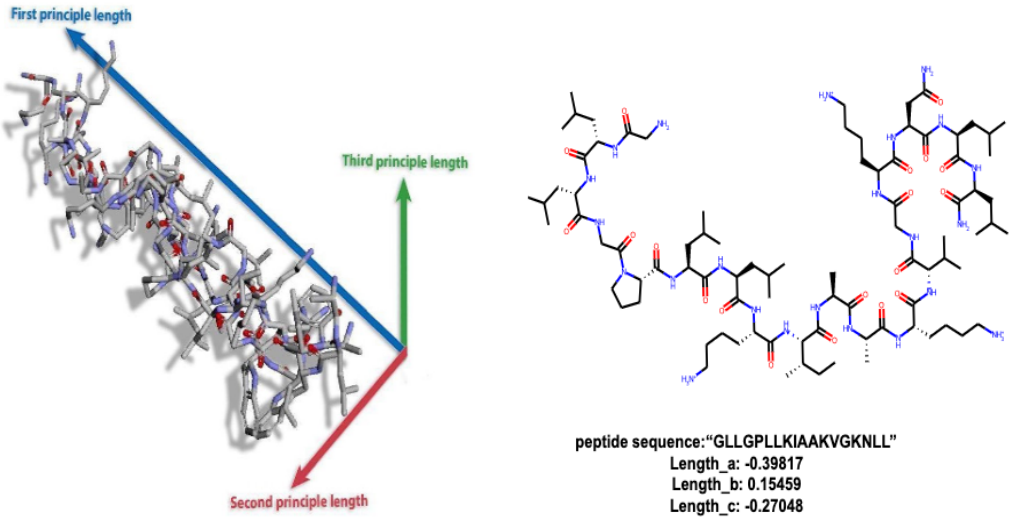
CPSC 440/550 Advanced Machine Learning (Jan-Apr 2025)

Optional Assignment 4 – due Saturday, April 25 at **11:59pm**; no late days

The assignment instructions are the same as for the previous assignments. If you do the assignment with a partner, please only hand in one copy for the group (using the appropriate Gradescope feature).

1 Graph Neural Networks [85 points]

While not nearly as common as tabular data, images, or natural language, another major modality of data that's seen a lot of interest particularly in the past decade is *graph* data. There are multiple related problem setups, but here we wish to learn a *function on graphs*: similar to how in image classification we might want to learn a function whose input is an image and output is a class label, here we want to learn a function whose input is the molecular graph of a peptide, and whose output gives some aggregate 3D properties of that molecule's structure, such as its length, "sphericity," and inertia.



This is the `Peptides-struct` dataset from the Long-Range Graph Benchmark. Each node represents a heavy atom in the peptide, and the (undirected) edges a molecular bond; nodes have pre-computed features associated with them. (There are also edge features in this dataset, but we're ignoring them.)

We've pre-processed the dataset to include eigendecompositions of the graph Laplacian, which will be useful for constructing Laplacian positional encodings in the `LapPENodeEncoder` class in `gnns.py`. This gives some notion of "location" within the graph to each node, similar to the trigonometric position features we mentioned for sequence Transformers. (We won't use this for the GCN, though you could, and it would help.) You don't have to worry about this, but you can check out that class (and the code inside the `if False` block in `gnn_utils.py`) if you're curious.

To work with graph data, you'll want to install the PyTorch Geometric library, with `pip install torch_geometric`. You won't have to worry about any internals here, but some of its helpers will handle some of the grunt work of working with this kind of data. The first time you run `main.py gcn`, it'll download the dataset into the `data` folder; it's about 400 MB, so I didn't put it in the zip.

This dataset is bigger than the datasets we've used before (ie MNIST); it's still runnable on a decent laptop, but it might be a little annoying, especially for the Transformer later. You might prefer to use a GPU on Google Colab (which is free), or some other machine with a CUDA-capable GPU. To use Colab, go to `https://colab.research.google.com`, and request a GPU under `Runtime` → `Change runtime type`. Then run `!pip install torch_geometric`, and open the Files tab on the left (the folder icon) and upload the contents of the code directory. Then, after `import main`, you can use either `main.train_gcn()` or `main.run("gcn")`. Be careful with editing files on Colab, though; unless you save them to your Google Drive or similar, they can just vanish on you! One thing you could do is write your code locally, make sure it runs on a batch or two, then upload to Colab for a final run to make sure it trains okay.

If you have CUDA set up for your GPU in pytorch, e.g. on Colab, it should use that automatically. You

can use `main.py use-cpu gcn`, `main.py use-mps gcn`, or `main.py use-cuda gcn` to force a device. MPS is the GPU on recent Macs; on my machine, I get pretty mixed results with CPU usually faster on this workload, but your mileage may vary. If you run into memory issues, try decreasing the batch size given to the data loader; if you have plenty of free memory, you could try increasing it to see if that's faster.

`main.py gcn` will run a Graph Convolutional Network on this data, using the `torch_geometric` library's implementation. This model is, while not state of the art or anything, "pretty okay" on this dataset. This dataset is pretty accessible by the standards of modern datasets; the code as-is runs an epoch in about ten seconds on my laptop's CPU (four on a Colab GPU). Here we only run three epochs out of laziness, but if you train longer it'll do better.

[1.1] [35 points] Implement your own graph convolution operation, rather than using PyTorch Geometric's; there's scaffolding for you in `MyGCNConv`. (`main.py my-gcn` runs that for you; you shouldn't need to change anything in the `GCN` class.) A graph convolution is given by

$$\mathrm{GCN}(x)_v = \sum_{u \in \text{neighbours}(v) \cup \{v\}} \frac{1}{\sqrt{\deg(v)\deg(u)}} W x_u + b,$$

where v and u are nodes in the graph with corresponding node feature vectors x_v and x_u. (Recall that a neighbour of node v is any node with an edge from v; the degree of a node is its number of neighbours.) The parameters of the matrix are the matrix W of shape `[dim_out, dim_in]` and the bias vector b of shape `[dim_out]`.

While they implement it with a relatively complex message passing framework, do not use this in your code. You may want to use a matrix multiplication framing (you probably don't want to do explicit looping). Your implementation will likely be slower than theirs, but run it for a few epochs to make sure the prediction error is about the same; my straightforward matrix-multiplication implementation takes about 40 seconds per epoch (instead of 10) on my laptop's CPU, or about 7 seconds (instead of 4) on a Colab GPU. The accuracy might not be exactly the same (there's a bunch of randomness, and a few things will be slightly different); as long as it's going down and not a huge amount higher than the other implementation, it should be fine. Hand in your code.

Answer:

```python
class MyGCNConv(nn.Module):
    def __init__(self, dim_in, dim_out):
        super().__init__()

        self.dim_in = dim_in
        self.dim_out = dim_out

        self.weight = torch.nn.Parameter(torch.empty((dim_out, dim_in)))
        self.bias = torch.nn.Parameter(torch.empty(dim_out))
        self.reset_parameters()

    def reset_parameters(self):
        # the scheme used by torch_geometric
        nn.init.xavier_uniform_(self.weight)
        nn.init.zeros_(self.bias)

    def forward(self, x, edge_index):
        # x is an array of node features, shape [nodes_in_batch, dim_in]
        # edge_index is an array of edges, shape [2, edges_in_batch]
        #   each edge goes from node edge_index[0, i] to edge_index[1, i]
```

```
        #   this dataset has undirected edges, which means each edge is in here
        ↪  twice

        # add "self-loops" (edges from v to v) for each node
        edge_index, _ = add_self_loops(edge_index, num_nodes=x.shape[0])
        deg = degree(edge_index[1], x.shape[0], dtype=x.dtype)  # shape
        ↪  [nodes_in_batch]

        adj = to_dense_adj(edge_index).squeeze(0)

        inv_sqrt_deg = deg.clamp_min(1).pow(-0.5)
        norm_adj = adj * inv_sqrt_deg.unsqueeze(1) * inv_sqrt_deg.unsqueeze(0)

        x = norm_adj @ x
        x = F.linear(x, self.weight, self.bias)
        return x
```

[1.2] [50 points] While there are many variants of Transformers for graph data, the class `gnns.GraphTransformer` implements a simple variant that does pairwise attention over all nodes in the graph. Similarly to how the sequence order of a sentence is only available to a standard Transformer via positional encodings, in this version of a graph Transformer, the structure of the graph is only available via the Laplacian positional encoding. While this can cause issues on some datasets, in this dataset it turns out to be pretty okay. `main.py graph-transformer` runs this model using torch's implementation of multi-headed self-attention.

Because of the additional pairwise attention, this method is slower than GCNs, so if you don't have a decent GPU it'll really be nicer to train on Colab (where an epoch takes 15 seconds, instead of about 5 minutes on my CPU!). But you can develop locally, make sure your code runs for a couple of batches, and then try training on Colab. (If you want, try training for longer to get a much better predictive model!)

Finish the implementation of the `gnns.MultiHeadSelfAttention.forward()` method; you can run it with `main.py my-graph-transformer`. My implementation is again slower than the pytorch implementation (which has had a lot of optimization effort put into it), but regression performance is the same. Make sure it runs okay, and hand in your code.

Answer:

```
def forward(self, x):
    # assume x is a torch.nested.nested_tensor
    #  x[graph_i, node_i, j] is the jth feature of the node_ith node from graph_i
    n_graphs = x.size(0)
    #  x.size(1) is irregular and would throw an error
    #  x.size(2) is the feature dimension == self.dim_in

    q = self.map_Q(x)  # [n_graphs, <irregular>, n_heads * dim_qk]
    k = self.map_K(x)  # [n_graphs, <irregular>, n_heads * dim_qk]
    v = self.map_V(x)  # [n_graphs, <irregular>, n_heads * dim_v]

    # split things up per head
    # for nested_tensors, -1 here means "stay the same"
    # so we end at [n_graphs, n_heads, <irregular>, dim_*]
    q = q.reshape(n_graphs, -1, self.n_heads, self.dim_qk).transpose(2, 1)
    k = k.reshape(n_graphs, -1, self.n_heads, self.dim_qk).transpose(2, 1)
```

```
        v = v.reshape(n_graphs, -1, self.n_heads, self.dim_v).transpose(2, 1)

        # scaled dot-product attention
        scores = (q @ k.transpose(-2, -1)) / math.sqrt(self.dim_qk)
        attn = F.softmax(scores, dim=-1)
        attn = self.dropout(attn)

        x = (attn @ v).transpose(1, 2)
        x = x.reshape(n_graphs, -1, self.n_heads * self.dim_v)

        # at this point, x has shape [n_graphs, <irreg>, n_heads * dim_v]
        x = self.map_out(x)  # [n_graphs, <irreg>, dim_out]
        return x
```

2 Challenge problem [15 points]

Describe a tractable family of distributions that is universal for approximation in reverse KL. By "tractable," I mean that there should be algorithms that take time and space polynomial in the data dimension such that, given a specification of any q in the family, you can (a) output $q(x)$ for any x and (b) obtain an exact sample distributed according to q. By "universal," I mean that for any target distribution p and any $\varepsilon > 0$, there is a q in this family such that $\mathrm{KL}(q\|p) < \varepsilon$. For partial credit (10/15 points), you may give a family universal in the space of distributions with strictly positive densities (wrt Lebesgue measure) and finite differential entropy.

Hint: One route to doing this could be to prove that #P (a superset of NP) is equal to P; I'm pretty sure there's an answer then.

Answer: TODO